base. This React component is built using Next.js's app router with client-side interactivity and uses the Vercel AI SDK's hooks for managing conversational state.

With the core components and services thoroughly explained, we now shift our focus to the key challenges and critical considerations encountered during the development of this web application, along with proposed future improvements and architectural enhancements aimed at ensuring scalability, security, and maintainability.

11.2 Challenges during development

Building this RAG web application introduced a range of challenges, some similar to those faced in the AI interview assistant, and others unique to the complexities of managing multiple knowledge bases, documents, and vector search.

11.2.1 Shared vs. dedicated user data in vector stores

When storing vectorized document embeddings, a fundamental architectural decision is whether to provision a dedicated vector store instance for each user or to consolidate all users' embeddings within a shared database. This choice directly affects data isolation, query performance, operational complexity, and cost.

In our application, we chose to use a shared Upstash Vector database with strict namespacing and metadata filtering based on the knowledge base and user identifiers. This approach optimizes resource utilization and simplifies infrastructure management while maintaining logical data isolation through query-level filters.

However, in ideal scenarios, especially where stringent security or regulatory compliance is required, the customer data should be physically separated. Provisioning isolated vector databases per user or tenant enhances data privacy and reduces risk but with increased operational overhead and cost.

Implementing fine-grained access control policies at the vector store layer is crucial as well. These controls prevent cross-user data leakage by enforcing strict authorization on all vector search and storage operations.

11.2.2 Security considerations around document management and heavy workloads

Uploading, parsing, and vectorizing documents are resource-intensive operations that also carry potential security risks due to the sensitive nature of document contents. Ensuring the secure handling of files and protecting backend services from abuse is therefore critical.

To address this, we integrated Clerk.js for authentication, ensuring that only authorized users can upload or access documents. File uploads undergo validation on both the client and server sides to enforce allowed file types and size limits. Document processing (parsing and chunking) occurs securely on the server with error handling to prevent crashes and maintain system stability.

However, these measures are only the beginning. To further enhance security and reliability, several improvements should be considered: